

TypeScript

Typescript

- Typescript è un linguaggio di programmazione creato da Microsoft per superare alcuni problemi e limitazioni di javascript.
- E' stato creato nel 2012 ed oggi è arrivato alla versione 5.9 (31/07/2026)
- E' rilasciato con licenza Open Source
- Typescript è un superset di javascript
 - Qualsiasi programma javascript formalmente valido è anche un programma typescript valido
 - Typescript aggiunge alcune caratteristiche in più per renderlo più robusto e per generare codice più manutenibile

Typescript

- Le caratteristiche principali di typescript sono:
 - Il linguaggio è "compilato" in javascript:
 - Non può essere eseguito direttamente
 - La compilazione può intercettare errori
 - Tipizzazione statica (come C, C++, C#, Java...) → Typescript è javascript con il controllo dei tipi
 - Possibilità di aiuti alla codifica da parte dell'editor (Intellisense)
 - Regole sintattiche più rigide (e sicure) rispetto a typescript
 - Utilizzato da alcuni framework di sviluppo web (p.e. Angular)
 - Adatto ad applicazioni medio-grandi

Requisiti per l'utilizzo di typescript

- Node.js
- Typescript compiler
 - `npm i -g typescript`
 - `tsc -v`
- Code editor (Visual Studio Code consigliato)

Typecript Hello World

- Ogni file typescript deve avere l'estensione .ts
- Esempio: HelloWorld.ts
 - `console.log('Hello World');` //Valido in quanto codice js
- Compilazione `tsc HelloWorld.ts` → generando HelloWorld.js
- Esecuzione di HelloWorld.js
 - Da browser
 - Con node

TypeScript Hello World

Esempio con codice typescript:

- Test.ts
 - `let x : number = 10;`
- Compilazione `tsc Test.ts` → generando Test.js
- Test.js contiene
 - `var x = 10;`

NB: Per default, il compilatore genera codice js che non utilizza le ultime funzionalità ES6.

Configurare il compilatore

E' possibile configurare il comportamento del compilatore, generando un file chiamato `tsconfig.json`. Può essere creato direttamente con `tsc`: `tsc --init`

Il file generato contiene quasi tutti i settaggi disponibili, disabilitati per default.

La presenza di `tsconfig.json` permette di compilare semplicemente con il comando `tsc`

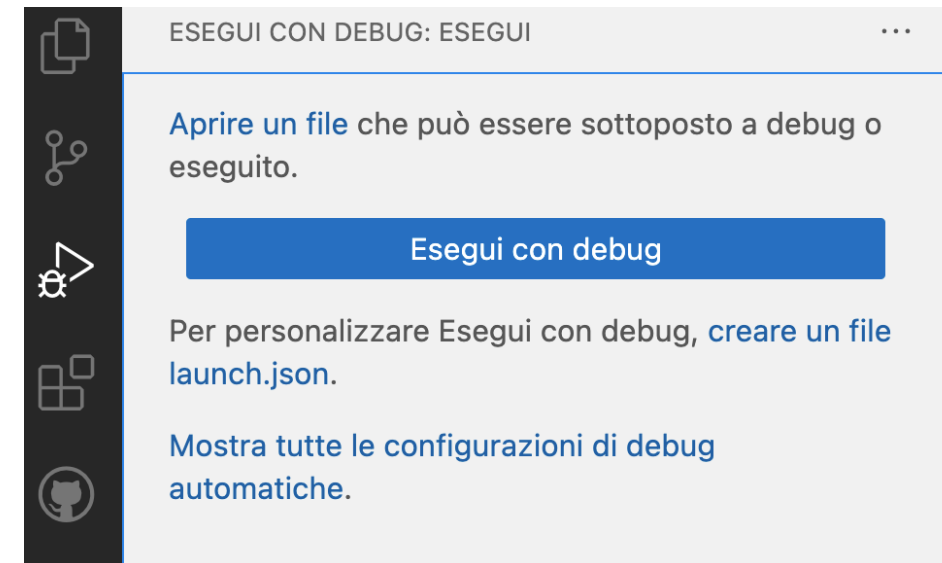
Alcune voci interessanti:

- `Target`: Versione di compatibilità del js generato
- `rootDir`: Posizione dei file ts
- `outDir`: Posizione dei file js generati
- `removeComments`: Per eliminare i commenti dai file js
- `noEmitOnError`: Per bloccare la generazione del js in caso di errori nel ts

Debuggare programmi typescript

Il debug può essere fatto direttamente con Visual Studio Code. Il programma typescript deve però essere configurato opportunamente:

1. Attivando la voce `"SourceMap"` : `true` in `tsconfig.json`
2. Creare un file di debug `launch.json` dal menù debug di VsCode, scegliendo `node.js` come compilatore
come compilatore
3. Utilizzare i comandi messi a disposizione da VsCode



Alcune caratteristiche di base

- Typescript, analogamento a javascript, è un linguaggio **case sensitive**. C'è differenza tra lettere maiuscole e minuscole
- Tutte le istruzioni devono terminare con il carattere ;
- I blocchi di codice sono identificati da una coppia di parentesi graffe { }
- I commenti sono specificati in due modi differenti:
 - I commenti di riga iniziano con //
 - I blocchi di commenti, che si possono estendere su più righe, sono delimitati dalla coppia di caratteri /* e */

Dichiarare variabili e costanti

```
let variabile : number;
```

```
let variabile : number = 10;
```

```
let variabile = 10;
```

```
const Costante : number = 12;
```

Identificatori

Gli identificatori utilizzati per i nomi di variabili e di costanti devono rispettare le seguenti regole:

1. Non possono iniziare con un numero
2. Non possono contenere spazi
3. Non possono contenere parole riservate, per esempio `int`. Eventualmente anteporre un carattere speciale, quale `@`

E' inoltre opportuno che i nomi di variabili e costanti:

1. Utilizzino nomi significativi
2. Utilizzino una naming convention
 1. `camelCase` (per variabili)
 2. `PascalCase` (per costanti)

Tipi di dati primitivi

In typescript possiamo usare tutti i dati primitivi previsti da js:

- number
- string
- boolean
- null
- undefined
- object

Tipi di dati primitivi

Typescript inoltre aggiunge altri tipi di dati primitivi

- any
- unknown
- never
- tuple
- enum

Tipi di dati primitivi - any

- any permette di definire una variabile a tipizzazione debole, come in js.
 - `let x : any;`
 - `let x; //dedotto implicitamente`
- L'utilizzo è sconsigliato e deve essere limitato a casi particolari
- La deduzione implicita di any può essere disabilitata in tsconfig.json:
 - `"noImplicitAny" : true`

Tipi di dati primitivi - unknow

- unknown permette di definire un parametro di tipo sconosciuto senza avere la libertà di any
 - `let x : unknown;`
- L'utilizzo è possibile dopo averne effettuato il type narrowing:
 - `if(typeof x === "string") { //Uso x come una stringa }`
 - `if(typeof x === "Date") { //Uso x come un Date }`

Tipi di dati primitivi - never

- never permette di definire una funzione che non restituisce mai il controllo al chiamante, per esempio a seguito di un ciclo infinito.
- Il compilatore utilizza questa conoscenza per segnalare istruzioni non raggiungibili perché successive alla chiamata di funzione senza terminazione

```
function infiniteLoop() : never
```

```
{
```

```
    while(true)
```

```
    { ... }
```

```
}
```

```
infiniteLoop();
```

```
console.log('Elaborazione terminata'); //ERRORE DI COMPILAZIONE
```


Array

- In typescript gli array contengono elementi tutti dello stesso tipo, che va specificato in fase di definizione:
 - `let numeri : number[];`
 - `let numeri = [1,2,3];`
- E' possibile, ma sconsigliato, creare array contenenti elementi di tipo differente. E' sufficiente creare un array di tipo any:
 - `let array : any[];`
 - `let array = [];`

Tipi di dati primitivi - tuple

- Le tuple sono strutture che raggruppano più dati di tipo differente. La sintassi fa uso di parentesi quadre come gli array:
- Creazione di una tupla di due elementi, un numero e una stringa:
 - `let articolo : [number, string] = [1, "Penna Rossa"];`
- Utilizzo del primo dato: `user[0]`
- Utilizzo del secondo dato: `user[1]`

NB: Le tuple sono tradotte in array javascript

Enumerazioni

Le enumerazioni sono strutture dati usate per assegnare valori mnemonici ad un set di numeri interi. Sono molto utili per due motivi:

- Aumentano di molto la leggibilità e la velocità di scrittura del codice. E' molto più facile riferirsi ad un giorno della settimana con il suo nome (lunedì, martedì...) piuttosto che con un numero.
- Aumentano la sicurezza del codice. Non è cioè possibile assegnare un valore non previsto. Cosa che potrebbe accadere utilizzando un intero o una stringa.

Le enumerazioni si definiscono con la parola chiave `enum` seguita dal nome della enumerazione e dall'elenco dei valori ammessi riportati tra parentesi graffe.

```
enum GiorniSettimana { Lunedì, Martedì, Mercoledì, Giovedì, Venerdì, Sabato, Domenica }
```

```
let oggi : GiorniSettimana = GiorniSettimana.Martedì;
```

Enumerazioni

Se non diversamente specificato, i valori dell'enumerazione vengono mappati a valori numerici interi. Il primo elemento dell'enumerazione viene mappato con zero, il secondo con uno e così via.

E' possibile specificare uno o più valori di mapping, così come è possibile specificare un valore non numerico, per esempio una stringa. In tal caso è però necessario associare un valore ad ogni elemento dell'enum.

```
enum GiorniSettimana { Lunedì = 1, Martedì, Mercoledì, Giovedì, Venerdì, Sabato, Domenica }
```

```
enum GiorniSettimana { Lunedì = 'Lu', Martedì='Ma', Mercoledì= 'Me', Giovedì = 'Gi', Venerdì = 'Ve', Sabato = 'Sa', Domenica='Do' }
```

Funzioni in typescript

In typescript è necessario definire il tipo di ogni parametro, così come il tipo del valore di ritorno.

In questo modo il compilatore è in grado di verificare l'utilizzo corretto della funzione, garantendo che vengano passati valori del tipo corretto e che venga restituito un valore di tipo adeguato.

```
function calcolaValoreIvaCompresa( valoreSenzalva:number ) : number
{
    return valoreSenzalva * 1.22;
}
```

NB: Se non viene specificato il tipo di ritorno, questo viene dedotto dal tipo restituito con return. Se ci sono più return che restituiscono tipi differenti viene dedotto un valore di ritorno di tipo any → Specificare sempre il tipo di ritorno, utilizzando `void` per le funzioni che non devono ritornare nulla

Oggetti in typescript

In typescript gli oggetti non sono dinamici. Una volta dichiarati non possono essere modificati.

La dichiarazione deve definire il tipo di ogni campo, che può essere specificato in modo esplicito o dedotto in modo implicito:

```
let persona = { id:1, nome:'Marco Rossi' }
```

```
let persona : {  
  id:number,  
  nome:string  
} = { id:1, nome:'Marco Rossi' }
```

Oggetti - Campi Opzionali

Tutti i campi di un oggetto typescript devono essere inizializzati, altrimenti si genera un errore.

E' possibile, esplicitamente, definire campi opzionali, ovvero che non necessitano di inizializzazione. E' sufficiente inserire un ? dopo il nome del campo(? che non fa parte del nome).

Un campo opzionale, non inizializzato, vale undefined

```
let persona : {  
  id:number,  
  nome?:string  
} = { id:1 }  
  
console.log(persona.nome); //undefined
```

NB: utilizzare campi opzionali solo se strettamente necessario.

Oggetti - Campi ReadOnly

E' possibile definire un campo in sola lettura, che non può più essere modificato dopo che l'oggetto è stato creato. E' sufficiente inserire la parola chiave `readonly` prima del nome del campo.

```
let persona : {  
    readonly id:number,  
    nome:string  
} = { id:1, name:'Marco Rossi' }
```

```
persona.id = 2; //Errore
```


Oggetti - Metodi

E' possibile definire un metodo in modo molto snello tramite arrow functions:

```
let persona : {  
  readonly id:number,  
  nome:string,  
  calcolaEtà : (dataNascita: Date) => number  
} = {  
  id:1,  
  nome:'Mario Rossi',  
  calcolaEtà: (dataNascita: Date) => { ... }  
}
```

NB: Torneremo sull'argomento quando parleremo di OOP in typescript

Oggetti - Alias

E' possibile separare la definizione di un oggetto dalla sua implementazione, in modo da strutturare meglio il codice evitando duplicazioni. Con la parola chiave `type` creiamo un nuovo tipo di dato utilizzabile tutte le volte che ne abbiamo bisogno.

```
type Persona = {  
  readonly id:number,  
  nome:string,  
  calcolaEtà : (dataNascita: Date) => number  
}  
  
let persona : Persona = {  
  id:1,  
  nome:'Mario Rossi',  
  calcolaEtà: (dataNascita: Date) => { ... }  
}
```

Alias

Gli alias sono utilizzabili per tutti i tipi di dati, anche non oggetti.

```
type Anno = number;
```

```
let oggi : Anno = 2026;
```

Union types

In typescript è possibile definire una variabile che può assumere valori di più tipi di dati, per esempio una stringa e un numero. Per farlo è sufficiente specificare tutti i tipi di dati separati da |

L'operatore typeof permette poi di identificare il tipo reale che la variabile sta assumendo in quell'istante.

```
function kgToLibbre(peso : number | string) : number
{
  if( typeof peso === 'number')
    return peso * 2.2;
  else
    return parseInt(peso)*2.2;
}
```

Literal types

Definendo una variabile utilizzando valori al posto dei tipi di dato, è possibile limitare i valori che essa potrà acquisire.

Possiamo utilizzare l'operatore di union (|), così come creare degli alias types

```
let qta : 10 | 11 = 10;
```

```
type Qta = 10 | 11;
```

```
let qta : Qta = 10;
```

Nullable types

Per comportamento predefinito Typescript non accetta valori null, diversamente da quanto accade in javascript.

Per modificare il comportamento è necessario specificare, espressamente, la volontà di accettare il valore null, utilizzando l'operatore union (|).

Discorso analogo per undefined.

```
function kgToLibbre(peso : number | string) : number { ... }
```

```
kgToLibbre(null); //Errore
```

```
function kgToLibbre(peso : number | string | null) : number
```

```
{  
  if(peso) {...}  
}
```

```
kgToLibbre(null); //OK
```

Optional access operator

Se un oggetto vale null o undefined non è possibile accedere ai suoi campi o metodi. Prima di accedervi è necessario verificare che l'oggetto esista e sia `!= null` e `!= undefined`

```
function myFunction(id : number ) : MyObject | null { ... }  
let o = myFunction(1);  
if(o != null) console.log(o.field);
```

Un modo alternativo e più compatto è quello di usare l'operatore di accesso opzionale

```
console.log(o?.field);
```

In caso l'oggetto o valga null o undefined verrà restituito undefined senza generare alcun errore

Oggetti – Optional access operator

L'operatore di accesso opzionale può essere usato anche con array e funzioni:

- Prima di accedere ad un qualsiasi elemento di un array è necessario verificare che l'array non sia null o undefined

```
let MyArray : number[] | null | undefined;
```

```
let x = MyArray?[0]
```

- Prima di eseguire una funzione è necessario verificare che sia diversa da null o undefined:

```
let log : any = null;
```

```
log?.('a');
```


Colaescing operator

Supponendo di avere una variabile che può valere anche null.

Supponendo inoltre di voler assegnare tale valore ad un'altra variabile, settando però un valore di default se la prima vale null. Abbiamo più strade:

```
let varA : number | null = null;  
let varB : number = varA || 10; //NON FUNZIONA se varA vale 0
```

```
let varB : number = varA === null ? 10 : varA;
```

```
//Coalescing operator
```

```
let varB : number = varA ?? 10;
```

OOP in Typescript

OOP in typescript

Typescript ha un approccio alla programmazione ad oggetti analoga a quella di C#, riprendendone molti concetti, sintassi e parole chiave.

OOP in typescript si basa su:

- Classi
 - Metodi
 - Proprietà
 - Modificatori d'accesso
 - Getter e setter
 - Static
- Oggetti
- Ereditarietà
 - Override
- Polimorfismo

Classi

Una classe è definita con la parola chiave `class` e contiene una serie di proprietà, di metodi e di costruttori:

```
class Sensore {  
  
    //Prop  
    matricola: string;  
    temperature : number[];  
  
    //Costruttore  
    constructor(matricola:string, temperature:number[]) {  
        this.matricola = matricola;  
        this.temperature = temperature;  
    }  
  
    //Metodo  
    temperaturaMedia() : number { ... }  
}
```

Classi

Tutte le proprietà devono essere inizializzate ad un valore valido, impostando un valore di default oppure inizializzandole nel costruttore.

```
class Sensore {  
  
    //Prop  
    matricola: string;  
    temperature : number[];  
    prezzo : number; //ERRORE  
  
    //Costruttore  
    constructor(matricola:string, temperature:number[]) {  
        this.matricola = matricola;  
        this.temperature = temperature;  
    }  
}
```

Classi

La creazione di un oggetto avviene tramite l'operatore new e la chiamata al costruttore:

```
let s = new Sensore("s001",[10,10,12]);
```

Classi

In typescript non è possibile effettuare l'overload di metodi e costruttori.

E' però possibile definire molteplici firme dello stesso metodo/costruttore, fornendo poi una implementazione in grado di coprirle tutte.

```
class Sensore {  
  
    //Prop  
    matricola: string;  
    temperature : number[];  
  
    //Firma dei Costruttori  
    constructor(matricola: string);  
    constructor(matricola: string, temperature:number[]);  
  
    //Unica implementazione del costruttore  
    constructor(matricola:string, temperature:number[] = []) {  
        this.matricola = matricola;  
        this.temperature = temperature;  
    }  
}
```

Classi

E' possibile definire proprietà in sola lettura, ovvero assegnabili solamente all'interno del costruttore, tramite la parola chiave `readonly`.

E' inoltre possibile definire proprietà opzionali, ovvero che possono anche assumere il valore `undefined`, tramite l'operatore `?`

```
class Sensore {  
  
    //Prop  
    readonly matricola: string;  
    temperature? : number[];  
}
```


Classi

Metodi e proprietà sono, per default, pubblici, ovvero utilizzabili anche dall'esterno della classe e dei relativi oggetti.

In typescript esistono 3 modificatori d'accesso:

- `public`: visibile ed utilizzabile anche dall'esterno della classe
- `private`: visibile ed utilizzabile solamente dai metodi della classe e non dall'esterno
- `protected`: come `private`, ad eccezione delle classi derivate per cui è `public`

NB: Torneremo più avanti sul rapporto tra ereditarietà e modificatori

Classi

In typescript è possibile semplificare la definizione di proprietà e costruttori. La definizione delle proprietà può infatti essere fatta direttamente nel costruttore.

La seguente classe:

```
class Sensore {  
    readonly matricola: string;  
    temperature : number[];  
    constructor(matricola:string, temperature:number[]) {  
        this.matricola = matricola;  
        this.temperature = temperature;  
    }  
}
```

Può essere riscritta come:

```
class Sensore {  
    constructor(public readonly matricola:string,public temperature:number[]) { }  
}
```

Classi

Typescript prevede la possibilità di definire getter e setter per accedere in modo controllato a campi privati:

```
class Voto
{
    private _punteggio:number=0;

    get Punteggio() : number { return this._punteggio; }
    set Punteggio(valore:number)
    {
        if(valore > 30) this._punteggio = 30
        else          this._punteggio = valore
    }
}
```

Classi - static

Metodi, campi e proprietà possono essere contrassegnati con la parola chiave static per poterli utilizzare anche senza la creazione di un oggetto. Possono essere utilizzate direttamente dalla classe.

```
class Voto
{
    private _punteggio:number=0;

    public static readonly PunteggioMax = 30;
}

console.log('Il punteggio massimo è:'+Voto.PunteggioMax);
```

Classi - Ereditarietà

L'ereditarietà (inheritance) è una relazione tra due classi che permette ad una classe di ereditare il comportamento da un'altra classe, per poi estenderlo.

La relazione tra due classi è di tipo is a: se la classe B eredita dalla classe A possiamo dire che B is a A. Esempio:

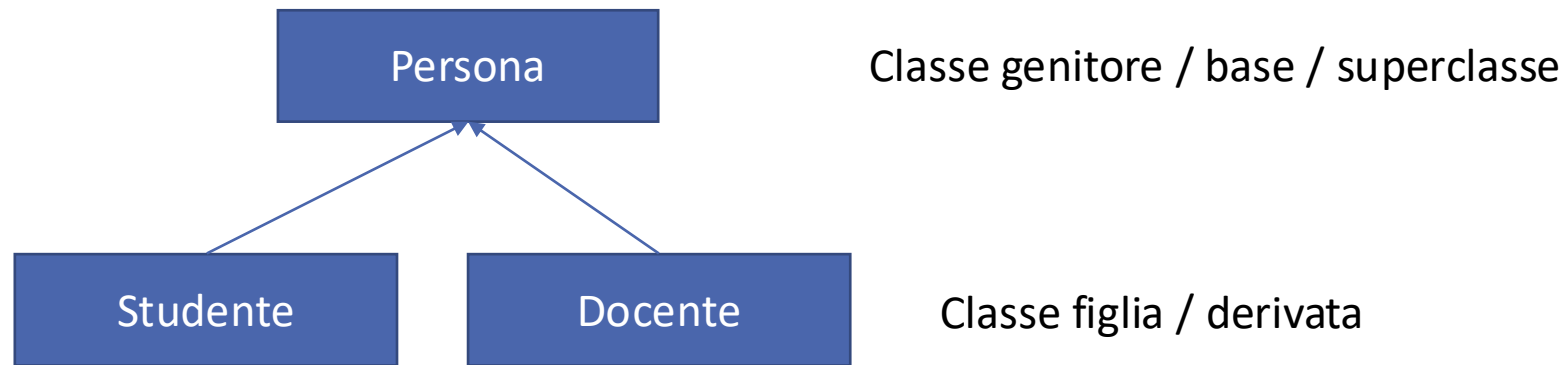
- Classe Persona
- Classe Studente che eredita da Persona → Studente is a Persona

Tramite l'ereditarietà abbiamo benefici in termini di:

- Riutilizzo del codice: implemento del codice in una classe e lo riutilizzo nelle classi che ereditano
- Polimorfismo: lo vedremo in seguito

Classi - Ereditarietà

L'ereditarietà tra classi può essere rappresentata graficamente:



NB: Una classe può ereditare al massimo da una superclasse.

Classi - Ereditarietà

Dal punto di vista sintattico l'ereditarietà avviene specificando il nome della classe genitore da cui si vuole ereditare:

```
class Persona
{
    constructor(public Cognome: string, public Nome:string, public CodiceFiscale : string ) { }
}
```

```
class Studente extends Persona
{
    public Matricola : string = "";
}
```

Classi - Ereditarietà

Se la sottoclasse non definisce un suo costruttore, allora eredita quello della superclasse. Eventuali proprietà definite nella sottoclasse devono pertanto avere un valore di default.

```
let s = new Studente('Rossi', 'Mario', 'MRRSS...');
```

Qualora la sottoclasse voglia creare un suo costruttore, questo dovrà richiamare quello della superclasse, utilizzando l'oggetto super

```
class Studente extends Persona
{
    constructor(Cognome: string, Nome:string, CodiceFiscale : string, public Matricola: string )
    { super(Cognome, Nome, CodiceFiscale); }
}
let s = new Studente('Rossi', 'Mario', 'MRRSS...', 'S001');
```


Classi – Ereditarietà e access modifiers

Una classe derivata NON può accedere alla parte private della classe base.

Metodi, campi e proprietà che utilizzano l'access modifier **protected** possono essere utilizzati all'interno della classe che li definisce (come private) ma anche all'interno delle classi derivate.

ATTENZIONE: Limitare l'utilizzo di protected solamente ai casi in cui è indispensabile. In tutti gli altri casi evitare di utilizzarlo, in quanto "espone" l'implementazione interna della superclasse.

Classi – Override dei metodi

Ci sono casi in cui una classe derivata necessita di modificare il comportamento di un metodo della classe base.

Il metodo mantiene la stessa firma (altrimenti sarebbe semplicemente un nuovo overload dello stesso metodo) ma implementa un comportamento differente.

Il metodo della classe derivata deve inoltre utilizzare la parola chiave `override` (se `noImplicitOverride:true`) e può utilizzare i metodi della superclasse tramite la parola chiave `super`

```
class Persona
{
    getFullName()
    {
        return this.Cognome + " " + this.Nome;
    }
}
```

```
class Studente extends Persona
{
    override getFullName()
    {
        return 'Stud ' + this.Cognome + " " + this.Nome;
    }
}
```

Classi – Polimorfismo

Il polimorfismo, che significa "multiforme", è il terzo pilastro della OOP, insieme all'incapsulamento e l'ereditarietà.

Grazie al polimorfismo, sfruttando la relazione di ereditarietà is a, oggetti di una classe derivata possono essere trattati come oggetti della classe base (o di una classe antenata)

```
class Persona
```

```
{  
  ...  
}
```

```
class Studente extends Persona
```

```
{  
  ...  
}
```

```
let p : Persona = new Studente('Rossi', 'Mario', 'MRRSS...', 'S001');
```

Classi – Polimorfismo

```
class Persona
{
    getFullName()
    {
        return this.Cognome + " " + this.Nome;
    }
}

class Studente extends Persona
{
    override getFullName()
    {
        return 'Stud ' + this.Cognome + " " + this.Nome;
    }

    mediaVoti()
    {
        return 0;
    }
}

let p : Persona = new Studente('Rossi','Mario','MRRSS...','S001');
p.mediaVoti(); //Errore
p.getFullName(); //Cosa stampa?
```

Quando tratto un oggetto di una classe derivata come fosse di una classe base, grazie al polimorfismo, posso utilizzare solamente metodi e proprietà definiti nella classe base.

In caso di override viene però eseguito il codice della classe derivata.

Classi Abstract

Talvolta è necessario obbligare ad estenderla impedendo di istanziare direttamente oggetti. Le classi di questo tipo vanno identificate come abstract.

```
abstract class Libro
{
    get Prezzo(){ ... }
}

class LibroDigitale extends Libro
{
}
```

Classi Abstract

Nelle classi abstract è possibile contrassegnare come abstract anche singoli metodi.

In tal caso si rende obbligatorio l'override nelle classi derivate e non è necessario fornirne l'implementazione.

```
abstract class Libro
{
    get Prezzo(){ ... }
    abstract get Info() : string;
}

class LibroDigitale extends Libro
{
    override get Info(): string { ... }
}
```

Classi – Interfacce

La creazione di una classe astratta pura, composta solo da metodi abstract, può essere utilizzata come una sorta di "contratto". Specifica infatti solamente la firma che le classi derivate devono rispettare (ed estendere) senza però determinare alcun comportamento.

Per rappresentare meglio questo comportamento esiste il concetto di interfaccia.

L'interfaccia permette di specificare metodi, campi e proprietà che una classe deve implementare se decide di aderire all'interfaccia. Ci sono due grandi vantaggi rispetto alle classi abstract:

- Classi che non sono in relazione di ereditarietà diretta possono implementare la stessa interfaccia (e quindi possono essere considerate omogenee rispetto all'interfaccia)
- Una classe può aderire a più interfacce contemporaneamente

Classi – Interfacce

Le interfacce si definiscono utilizzando la parola chiave `interface`. E' consuetudine farne iniziare il nome con una `I` maiuscola in modo da meglio identificarle.

Una classe può aderire ad una interfaccia specificandola in maniera analoga alla classe base da cui ereditare (eventualmente separando con virgole)

```
interface IGetInfo
{
    getInfo():string;
}
```

```
class LibroCartaceo extends Libro implements IGetInfo
{
    getInfo(): string { ... }
}
```

```
class Professore extends Persona implements IGetInfo
{
    getInfo(): string { ... }
}
```

```
let elenco : IGetInfo[];
```


Programmazione generica

In applicazioni reali sono frequenti casi in cui abbiamo proliferazione di codice "identico" che lavora però su tipi di dati differenti. E' il caso tipico delle strutture dati.

```
class Libreria
{
    private _datacCreazione : Date;
    private _dati: Libro[] = [];

    get count():number
    {
        return this._dati.length;
    }

    constructor()
    {
        this._datacCreazione = new Date();
    }

    Add(nuovo : Libro) : void
    {
        this._dati.push(nuovo);
    }
}
```

```
class Registro
{
    private _datacCreazione : Date;
    private _dati: Studente[] = [];

    get count():number
    {
        return this._dati.length;
    }

    constructor()
    {
        this._datacCreazione = new Date();
    }

    Add(nuovo : Studente) : void
    {
        this._dati.push(nuovo);
    }
}
```

Programmazione generica

Con la programmazione generica è possibile scrivere codice facendo riferimento a tipi di dati generici (astratti, non reali), specificando il tipo di dato reale solamente in fase di

utilizzo

```
class ArraySolaAggiunta<T>
```

```
{  
    private _dataCreazione : Date;  
    private _dati: T[] = [];  
  
    get count():number  
    {  
        return this._dati.length;  
    }  
  
    constructor()  
    {  
        this._dataCreazione = new Date();  
    }  
  
    Add(nuovo : T) : void  
    {  
        this._dati.push(nuovo);  
    }  
}
```

—————> Specifico che la classe che all'interno della classe verrà utilizzato un tipo generico chiamato T

Utilizzo il tipo T come se fosse un tipo reale

```
let Registro2 = new ArraySolaAggiunta<Studente>();  
let Libreria2 = new ArraySolaAggiunta<Libro>();
```

Programmazione generica

La specifica del tipo di dato generico può avvenire sulla classe (e quindi essere utilizzabile ovunque all'interno di essa) o sul singolo metodo (e quindi essere utilizzabile solo su di esso).

Può anche essere utilizzata in funzioni e in interfacce.

```
function reflect<T>(value : T) : T
{
    return value;
}
```

```
interface IGetInfo<T>
{
    getInfo():T;
}
```

```
class Utils
{
    static reflect<T>(value : T) : T
    {
        return value;
    }
}
```

Programmazione generica

Talvolta è necessario inserire dei vincoli sulla tipologia di dati utilizzabili al posto dei dati generici.

Questo può essere fatto utilizzando la parola chiave `extends` seguita dai tipi di dato, dalla classe o dall'interfaccia che il tipo di dato deve implementare.

```
function reflect<T extends number | string>(value : T) : T { return value; }
```

```
function reflect<T extends { name:string }>(value : T) : T { return value; }
```

```
function reflect<T extends IGetInfo>(value : T) : T { return value; }
```

```
function reflect<T extends Persona>(value : T) : T { return value; }
```

Moduli

Typescript prevede il concetto di "modulo" per l'organizzazione del codice:

- Il codice può essere diviso in più file chiamati moduli.
- Una classe definita in un modulo, per default, è utilizzabile solamente all'interno di quel modulo
- Un modulo può "importare" un altro modulo per utilizzarne il contenuto
 - Un modulo può importare solamente gli oggetti espressamente esportati dall'altro modulo

```
export class Cerchio
```

• L'esempio che segue riporta il contenuto di un modulo nel quale sono esportate due

```
classi  
{  
  constructor(public raggio:number){}  
}
```

```
export class Quadrato
```

```
{  
  constructor(public lato:number){}  
}
```

Moduli

L'import di un modulo avviene con la parola chiave `import`, specificando il nome del modulo da importare:

- Il nome del modulo va espresso con un percorso relativo rispetto al file nel quale si sta importando
- Non è necessario specificare l'estensione `.ts` (tipicamente il file ha estensione `.module.ts`)
- Sono possibili più sintassi per specificare cosa importare dal modulo
 - `import {Cerchio} from './forme.module';` → Import di una classe
 - `import {Cerchio as Circle} from './forme.module';` → Import di una classe con aliasing
 - `import {Cerchio, Quadrato} from './forme.module';` → Import di due classi
 - `import * as Shapes from './forme.module';` → Import di tutte le classi inserendole in un contenitore chiamato `Shapes`

Moduli

All'interno di un modulo è possibile identificare un oggetto come l'oggetto esportato di default.

Questo permette di importare quell'oggetto con una sintassi più snella che non fa uso di parentesi graffe. Parentesi che tornano ad essere necessarie se si decide di importare uno degli altri oggetti esportati.

```
export default class Cerchio
{
  constructor(public raggio:number){}
}
```

```
export class Quadrato
{
  constructor(public lato:number){}
}
```

`import Cerchio from './forme.module';` → Import della sola classe di default

`import Cerchio {Quadrato} from './forme.module';` → Import della classe di default più una seconda classe

Moduli

Per una migliore manutenzione del codice ogni classe deve risiedere in un file separato.

Pertanto il modulo sarà un file nel quale vengono importate tutte le classi (che devono avere la direttiva `export` nella loro definizione) e subito esportate:

```
import Cerchio from './cerchio';  
import Quadrato from './quadrato';  
  
export {Cerchio, Quadrato};
```


Decorators

Decorator sono attributi che applichiamo alle classi per modificarne il comportamento.

Sono molto utilizzati in Angular per, per esempio, trasformare una classe in un componente che può essere visualizzato sull'interfaccia utente.

Tecnicamente sono funzioni che prendono in input la classe su cui sono applicati e restituiscono una classe modificata, per esempio aggiungendo o rimuovendo proprietà/metodi.

Typescript non definisce alcun decoratore. Vanno creati manualmente o vanno utilizzati quelli creati in librerie/framework di terze parti (come Angular).

L'utilizzo è molto semplice mentre è più complicata la creazione (non la vedremo)

```
@Component
```

```
class Utils
```

```
{
```

```
...
```

```
}
```